

Personal Finance Tracker

SQL / MySQL Project Documentation

Project Type: Data Analyst / SQL Portfolio Project

Database: MySQL

Project Focus: Personal financial transaction tracking, expense analysis, savings analysis, and budget monitoring.

1. Project Overview

The Personal Finance Tracker is a relational database project developed using MySQL. The system stores users, financial accounts, income and expense transactions, categories, and monthly budgets. SQL queries are used to analyze financial behavior, calculate income, expenses and savings, compare budgets with actual spending, and identify spending patterns.

2. Project Objectives

- Design a normalized relational database for personal finance management.
- Maintain relationships between users, accounts, transactions, categories, and budgets.
- Track income and expenses using structured transaction records.
- Calculate total income, total expenses, and savings for each user.
- Analyze spending by category and identify high-value transactions.
- Compare monthly budgets with actual spending.
- Use advanced SQL concepts for analytical reporting.
- Implement reusable database objects such as views and stored procedures.
- Use a trigger to prevent invalid transaction amounts.

3. Database Structure

Table	Purpose	Important Columns
users	Stores user information	user_id, name, email, created_at
accounts	Stores bank accounts, cash, wallets and credit cards	account_id, user_id, account_name, account_type, balance
categories	Stores income and expense categories	category_id, category_name, category_type
transactions	Stores all financial transactions	transaction_id, user_id, account_id, category_id, amount, transaction_date
budgets	Stores monthly category budgets	budget_id, user_id, category_id, budget_amount, budget_month

4. Relationships

The database uses primary keys and foreign keys to maintain referential integrity. One user can have multiple accounts, transactions, and budgets. Each transaction is associated with a user, account, and category. Each budget is associated with a user and a category.

Relationship summary:

Users → Accounts

Users → Transactions ← Categories

Users → Budgets ← Categories

5. SQL Concepts Implemented

Concept	How it was used
DDL	CREATE DATABASE, CREATE TABLE, ALTER/structure-related operations
DML	INSERT records into users, accounts, categories, transactions and budgets
DQL	SELECT queries for analysis and reporting
JOINS	Combined users, transactions, accounts and categories
GROUP BY	Aggregated income, expenses and category spending
HAVING	Filtered grouped financial results
Subqueries	Compared transactions and user expenses with calculated values
CTEs	Simplified multi-step analytical queries
CASE	Classified transactions and calculated income/expense values
Window Functions	RANK, DENSE_RANK and running totals
Views	Created reusable transaction and financial summary views
Stored Procedures	Created parameterized user transaction and financial summary procedures
Triggers	Prevented invalid/non-positive transaction amounts
Aggregate Functions	SUM, AVG, MAX, COUNT

6. Key Analytical Queries

Total Income

```
SELECT SUM(t.amount) AS total_income
FROM transactions t
JOIN categories c ON t.category_id = c.category_id
WHERE c.category_type = 'Income';
```

Total Expense

```
SELECT SUM(t.amount) AS total_expense
FROM transactions t
JOIN categories c ON t.category_id = c.category_id
WHERE c.category_type = 'Expense';
```

Savings by User

```
SELECT u.name,
SUM(CASE WHEN c.category_type='Income' THEN t.amount ELSE 0 END) AS total_income,
SUM(CASE WHEN c.category_type='Expense' THEN t.amount ELSE 0 END) AS total_expense
FROM users u
JOIN transactions t ON u.user_id=t.user_id
JOIN categories c ON t.category_id=c.category_id
GROUP BY u.user_id, u.name;
```

Highest Spending Category

```
SELECT c.category_name, SUM(t.amount) AS total_spending
FROM transactions t
JOIN categories c ON t.category_id=c.category_id
WHERE c.category_type='Expense'
GROUP BY c.category_id, c.category_name
ORDER BY total_spending DESC
LIMIT 1;
```

Expense Ranking

```
RANK() OVER (ORDER BY ue.total_expense DESC) AS expense_rank
```

7. Budget Analysis

The budget analysis compares each user's category-level budget with actual transactions for the corresponding month. COALESCE is used so categories with no transactions return zero spending. A CASE expression classifies each budget as Within Budget, Budget Fully Used, or Budget Exceeded.

The project also calculates budget utilization percentage using: **Actual Spending / Budget Amount × 100**.

8. Views

transaction_details — combines transaction, user, account, and category information into a reusable view.

user_financial_summary — provides total income, total expense, and savings for each user.

9. Stored Procedures

GetUserTransactions(user_id) retrieves all transactions for a selected user. **GetFinancialSummary(user_id)** calculates the selected user's total income, total expense, and savings.

Example:

```
CALL GetUserTransactions(1);  
CALL GetFinancialSummary(1);
```

10. Trigger

The project includes a BEFORE INSERT trigger named **prevent_negative_transaction**. It checks NEW.amount and raises an error when the amount is less than or equal to zero. This adds an additional layer of data validation.